LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL [No. 04]

SINGLY LINKED LIST (BAGIAN PERTAMA)



Disusun oleh:

JANICA PRIMA GINTING

109082500064

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL GUIDE

Link Repo Github Modul 02 :
(https://github.com/Janica13/reddemon_StrukturData/tree/main/Modul%2004)

1. **Soal 1**

   Source Code

```
#include <iostream>

using namespace std;

typedef int infotype;
typedef struct elmlist *address;

struct elmlist {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L) {
    L.first = NULL;
}

address alokasi(infotype x) {
    address P = new elmlist;
    P->info = x;
    P->next = NULL;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = NULL;
}

void insertFirst(List &L, address P) {
    P->next = L.first;
    L.first = P;
}

void insertLast(List &L, address P) {
```

```
    if (L.first == NULL) {
        insertFirst(L, P);
    } else {
        address Q = L.first;
        while (Q->next != NULL) {
            Q = Q->next;
        }
        Q->next = P;
    }
}

void insertAfter(List &L, address P, address Prec) {
    if (Prec != NULL) {
        P->next = Prec->next;
        Prec->next = P;
    }
}

void deleteFirst(List &L, address &P) {
    if (L.first != NULL) {
        P = L.first;
        L.first = P->next;
        P->next = NULL;
    }
}

void deleteLast(List &L, address &P) {
    if (L.first != NULL) {
        if (L.first->next == NULL) {
            deleteFirst(L, P);
        } else {
            address Q = L.first;
            while (Q->next->next != NULL) {
                Q = Q->next;
            }
            P = Q->next;
            Q->next = NULL;
        }
    }
}

void deleteAfter(List &L, address &P, address Prec) {
    if (Prec != NULL && Prec->next != NULL) {
        P = Prec->next;
        Prec->next = P->next;
        P->next = NULL;
    }
}
```

```
void printInfo(List L) {
    address P = L.first;
    while (P != NULL) {
        cout << P->info << ' ';
        P = P->next;
    }
    cout << '\n';
}

int main() {
    List L;
    address P1, P2, P3, P4, P5;

    createList(L);

    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    printInfo(L);

    return 0;
}
```

Screenshoot program



Deskripsi program

Kode program ini adalah implementasi struktur data Singly Linked List modular dengan pendekatan pointer-to-struct dalam bahasa C++. Program ini mendefinisikan tipe data bentukan menggunakan typedef untuk infotype sebagai bilangan bulat dan address sebagai pointer penunjuk elemen, serta struktur List yang menampung pointer utama first

sebagai kepala penunjuk list. Program menyediakan sekumpulan fungsi dan prosedur standar untuk operasi manajemen memori dan manipulasi elemen, di antaranya createList untuk menginisialisasi list menjadi kosong, alokasi untuk memesan memori node baru, serta dealokasi untuk membersihkan memori node. Terdapat tiga variasi prosedur penyisipan data yaitu insertFirst (menyisipkan di awal), insertLast (menyisipkan di akhir), dan insertAfter (menyisipkan setelah node penunjuk tertentu), serta tiga variasi prosedur penghapusan data yaitu deleteFirst, deleteLast, dan deleteAfter. Selain itu, terdapat prosedur printInfo yang bertugas menelusuri list dari awal hingga akhir untuk mencetak nilai informasi setiap node ke layar. Di dalam fungsi main, program menginisialisasi list kosong, lalu mengalokasikan lima node secara berurutan dengan nilai 2, 0, 8, 12, dan 9 yang semuanya dimasukkan menggunakan metode insertFirst—sehingga elemen yang terakhir dimasukkan akan berada di posisi terdepan—yang kemudian diakhiri dengan pemanggilan prosedur printInfo untuk menampilkan seluruh isi elemen list tersebut.

LATIHAN SOAL MODULE

2. **Soal 1**

Source Code

```
#include <iostream>
#include <string>

using namespace std;

struct Node {
    string nama;
    Node* next;
};

Node* head = nullptr;
Node* tail = nullptr;

void insertLast(string namaBaru){
    Node* newNode = new Node();
    newNode->nama = namaBaru;
    newNode->next = nullptr;
    if (head == nullptr){
        head = newNode;
        tail = newNode;
    }
    else{
        tail->next = newNode;
```

```
            tail = newNode;
        }
    }

    void tampilkanAntrean(){
        if (head == nullptr){
            cout << "Antrean kosong" << endl;
            return;
        }
        cout << "Antrean: ";
        Node* curr = head;
        while (curr != nullptr){
            cout << curr->nama;
            if (curr->next != nullptr){
                cout << "->";
            }
            curr = curr->next;
        }
        cout << endl;
    }

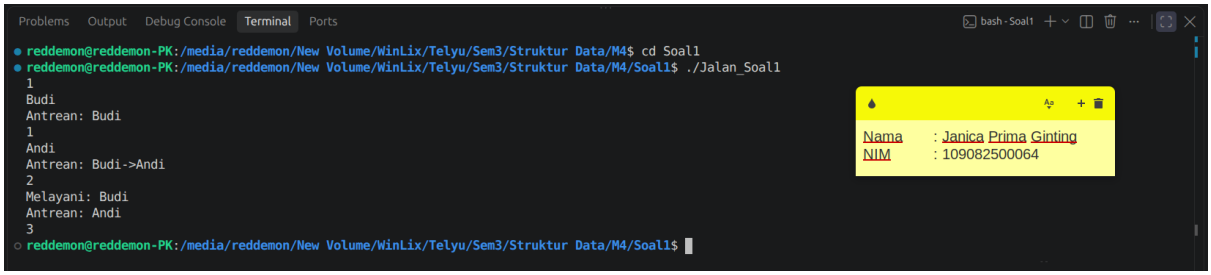
    void deleteFirst(){
        if (head == nullptr){
            cout << "Antrean kosong" << endl;
            return;
        }
        Node* temp = head;
        cout << "Melayani: " << temp->nama << endl;
        head = head->next;
        if (head == nullptr){
            tail = nullptr;
        }
        delete temp;
    }

    int main(){
        int pilihan;
        string nama;

        do{
            cin >> pilihan;
            if(pilihan == 1){
                cin >> nama;
                insertLast(nama);
                tampilkanAntrean();
            }else if (pilihan == 2){
                deleteFirst();
                tampilkanAntrean();
```

```
            }
        }while(pilihan != 3);
        return 0;
    }
```

Screenshoot program



Deskripsi program

Kode program C++ ini adalah program untuk mengimplementasikan Singly Linked List yang digunakan untuk menyimpan data antrean nasabah bank. Program ini memungkinkan pengguna untuk menambahkan nasabah baru ke dalam antrean menggunakan fungsi insertLast dengan memilih menu 1 dan memasukkan nama nasabah. Selain itu, program dapat melayani nasabah yang berada di urutan pertama menggunakan fungsi deleteFirst dengan memilih menu 2. Program berjalan secara terus-menerus menggunakan perulangan do-while hingga pengguna memilih untuk berhenti. Setiap kali ada perubahan pada antrean, program akan menampilkan status antrean saat ini secara dinamis dan menampilkan nama nasabah yang sedang dilayani.

3. Soal 2

Source Code <soal2.h>

```
#include <iostream>

using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* createNode(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = nullptr;
    return newNode;
}

void insertLast(Node*& head, int value) {
    Node* newNode = createNode(value);
```

```
    if (head == nullptr) {
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next != nullptr) {
        temp = temp->next;
    }
    temp->next = newNode;
}

void insertAfter(Node* head, int target, int newValue) {
    Node* temp = head;

    while (temp != nullptr && temp->data != target) {
        temp = temp->next;
    }

    if (temp != nullptr) {
        Node* newNode = createNode(newValue);
        newNode->next = temp->next;
        temp->next = newNode;
    }
}

void displayList(Node* head) {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data;
        if (temp->next != nullptr) {
            cout << " -> ";
        }
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    Node* head = nullptr;
    int n, target, newValue;

    if (!(cin >> n)) return 0;

    for (int i = 0; i < n; ++i) {
        int val;
        cin >> val;
        insertLast(head, val);
    }
```

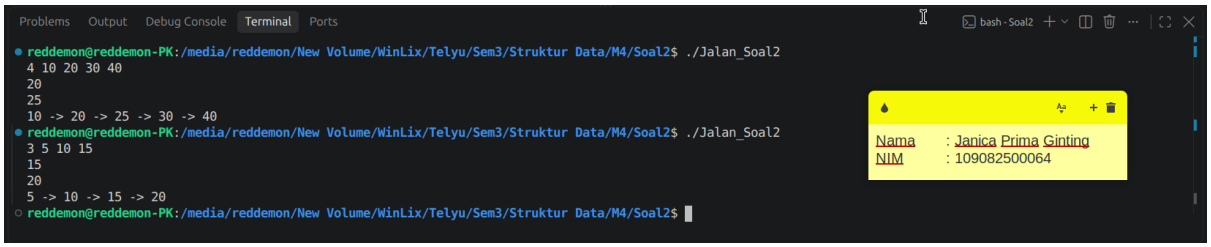
```
        cin >> target;
        cin >> newValue;

        insertAfter(head, target, newValue);

        displayList(head);

        return 0;
    }
```

Screenshoot program



Deskripsi program

Kode program ini adalah implementasi struktur data Singly Linked List menggunakan bahasa C++. Program ini diawali dengan pendefinisian struktur `Node` yang menyimpan nilai bilangan bulat pada variabel `data` dan pointer `next` yang menunjuk ke node berikutnya. Fungsi `createNode` bertugas untuk mengalokasikan memori dan menginisialisasi node baru dengan nilai tertentu. Terdapat prosedur `insertLast` yang digunakan untuk menambahkan elemen baru ke bagian akhir dari linked list guna membentuk rangkaian data awal. Selain itu, fungsi `insertAfter` digunakan untuk mencari node tertentu yang bernilai `target` di dalam list, lalu menyisipkan node baru berisi `newValue` tepat setelah node target tersebut ditemukan. Fungsi `displayList` berfungsi untuk menelusuri seluruh elemen dari awal hingga akhir sambil mencetak nilainya yang dipisahkan dengan tanda panah (`->`). Di dalam fungsi `main`, program membaca jumlah elemen awal `n` beserta nilai-nilainya dari pengguna untuk dimasukkan ke dalam list, membaca nilai `target` dan nilai baru yang akan disisipkan, menjalankan fungsi `insertAfter`, lalu menampilkan hasil akhir dari linked list ke layar.

4. Soal 3

Source Code

```
#include <iostream>

using namespace std;

struct Node {
    int data;
    Node* next;
```

```
};

Node* createNode(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = nullptr;
    return newNode;
}

void insertLast(Node*& head, int value) {
    Node* newNode = createNode(value);
    if (head == nullptr) {
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next != nullptr) {
        temp = temp->next;
    }
    temp->next = newNode;
}

int hitungJumlah(Node* head) {
    int total = 0;
    Node* temp = head;
    while (temp != nullptr) {
        total += temp->data;
        temp = temp->next;
    }
    return total;
}

int main() {
    Node* head = nullptr;
    int n;

    if (!(cin >> n)) return 0;

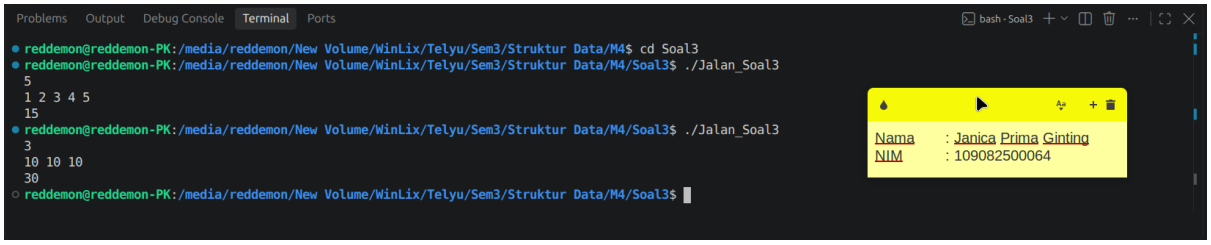
    for (int i = 0; i < n; ++i) {
        int val;
        cin >> val;
        insertLast(head, val);
    }

    int hasilJumlah = hitungJumlah(head);
    cout << hasilJumlah << endl;

    return 0;
```

```
}
```

Screenshoot program



Deskripsi program

Kode program ini merupakan implementasi struktur data Singly Linked List dalam bahasa C++ yang dirancang untuk menghitung total penjumlahan dari seluruh elemen di dalam list. Program ini diawali dengan pendefinisian struktur Node yang menampung variabel data bertipe integer dan pointer next untuk menunjuk ke node berikutnya. Fungsi createNode digunakan untuk mengalokasikan memori dan menginisialisasi node baru dengan nilai yang diberikan. Terdapat prosedur insertLast yang berfungsi untuk menyisipkan node baru ke posisi paling akhir dari linked list, di mana jika list masih kosong maka node baru akan langsung menjadi head. Selanjutnya, fungsi hitungJumlah bertugas menelusuri (traversal) seluruh linked list mulai dari head hingga akhir sambil mengakumulasikan nilai data dari setiap node ke dalam variabel total, lalu mengembalikan hasil penjumlahan tersebut. Di dalam fungsi main, program membaca jumlah elemen n dari pengguna, melakukan perulangan untuk mengisi list menggunakan insertLast, memanggil fungsi hitungJumlah untuk mendapatkan total keseluruhan nilai, dan mencetak hasil akhirnya ke layar.

5. Soal 4

Source Code

```
#include <iostream>
#include <string>

using namespace std;

struct Buku {
    string judul;
    int tahunTerbit;
};

struct Node {
    Buku data;
    Node* next;
};
```

```
Node* createNode(string judul, int tahunTerbit) {
    Node* newNode = new Node();
    newNode->data.judul = judul;
    newNode->data.tahunTerbit = tahunTerbit;
    newNode->next = nullptr;
    return newNode;
}

void insertLast(Node*& head, string judul, int tahunTerbit) {
    Node* newNode = createNode(judul, tahunTerbit);
    if (head == nullptr) {
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next != nullptr) {
        temp = temp->next;
    }
    temp->next = newNode;
}

void deleteLast(Node*& head) {
    if (head == nullptr) {
        return;
    }
    if (head->next == nullptr) {
        delete head;
        head = nullptr;
        return;
    }

    Node* temp = head;
    while (temp->next->next != nullptr) {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = nullptr;
}

void displayList(Node* head) {
    cout << "Daftar Buku:" << endl;
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data.judul << " (" << temp-
>data.tahunTerbit << ")" << endl;
        temp = temp->next;
    }
```

```
}

int main() {
    Node* head = nullptr;
    string line;

    while (getline(cin, line)) {
        if (line.empty()) continue;

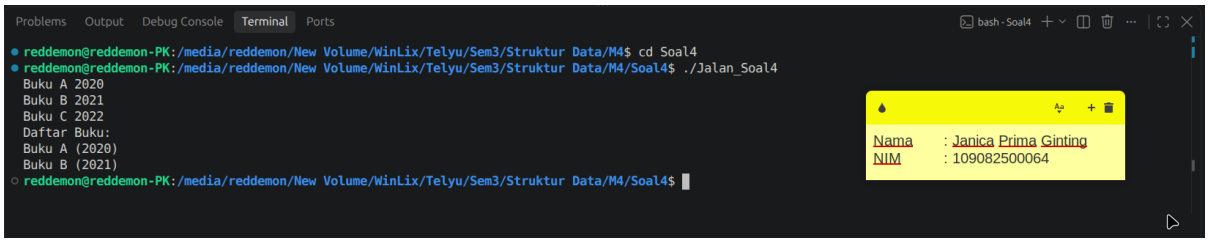
        size_t lastSpace = line.find_last_of(" ");
        if (lastSpace != string::npos) {
            string judul = line.substr(0, lastSpace);
            int tahunTerbit = stoi(line.substr(lastSpace +
1));
            insertLast(head, judul, tahunTerbit);
        }
    }

    deleteLast(head);

    displayList(head);

    return 0;
}
```

Screenshot Program



Deskripsi Program

Kode program ini merupakan implementasi struktur data Singly Linked List dalam bahasa C++ yang digunakan untuk mengelola data buku di perpustakaan dengan informasi berupa judul dan tahun terbit. Program ini mendefinisikan struktur Buku untuk menampung data judul dan tahun terbit, serta struktur Node yang membungkus data buku tersebut bersama pointer next untuk menghubungkannya ke node berikutnya. Terdapat fungsi createNode untuk mengalokasikan memori node baru dan prosedur insertLast guna menambahkan data buku di urutan paling akhir dari linked list. Prosedur deleteLast berfungsi untuk menghapus elemen buku yang berada di urutan paling akhir dari list, sementara prosedur displayList bertugas mencetak daftar sisa buku ke layar dengan format judul dan tahun terbit di dalam

tanda kurung. Di dalam fungsi main, program membaca input teks baris demi baris yang berisi judul dan tahun terbit secara dinamis, memasukkannya ke dalam linked list menggunakan insertLast, menjalankan operasi deleteLast sekali untuk menghapus buku terakhir, lalu menampilkan sisa daftar buku yang ada.